```
<session-config>
    <tracking-mode>COOKIE</tracking-mode>
</session-config>
```

This is the default in the Jetty container used by Spark. You can prevent session fixation attacks by ensuring that any existing session is invalidated after a user authenticates. This ensures that a new random session identifier is generated, which the attacker is unable to guess. The attacker's session will be logged out. Listing 4.8 shows the updated `CookieTokenStore`. First, you should check if the client has an existing session cookie by calling `request.session(false)`. This instructs Spark to return the existing session, if one exists, but will return `null` if there is not an existing session. Invalidate any existing session to ensure that the next call to `request.session(true)` will create a new one. To eliminate the vulnerability, open CookieTokenStore.java in your editor and update the login code to match listing 4.8.

Listing 4.8 Preventing session fixation attacks

```
    @Override
    public String create(Request request, Token token) {

        var session = request.session(false);    #A
        if (session != null) {    #A
            session.invalidate();    #A
        }
        session = request.session(true);    #B

        session.attribute("username", token.username);
        session.attribute("expiry", token.expiry);
```